웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

이 양식은 본인이 구현한 결과를 설명하기 위한 문서입니다. 대괄호 안내를 실제 내용으로 바꾸고 작성 안내 문장은 제출 전에 지우세요. 화면 캡처와 표의 행은 필요에 따라 추가할 수 있습니다. 6~10 쪽 내외를 권장하며 분량 자체는 평가하지 않습니다.

01 서비스 소개와 구현 범위

학번 202011711 이름: 이찬희 제출일: 26.10.28

서비스 이름 체크포인트

기술 조합과 버전 [React 또는 Vue / Express 또는 FastAPI 또는 Spring Boot / 실행 환경 버전] Vue + Express, npm 11.21.1

GitHub 저장소 URL https://github.com/ChanhuiL/Checkpoint

소스 코드 기준 main/Commit c1ea391

서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.
체크포인트는 게이머들을 위한 서비스이며, 각종 게임의 짤막한 팁이나 트리비아를 공유하는 서비스입니다.
프론트엔드로 Vue 를 사용한 이유는 이전에 사용해본 경험이 있기 때문이고, Express 의 경우 JS 기반 NestJS 를 사용해본 경험이 있기 때문에 조금이나마 더 익숙한 js 기반의 프레임워크를 사용하고 싶어 사용했습니다. Vue 의 경우 api.js 와 App.vue 를, Express 의 경우 auth.js 를 변경하였고 이 외의 파일들은 제공된 파일 그대로 사용합니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	express/store.js	
로그인과 JWT 발급	완료	express/auth.js	
인증과 내 정보 조회	완료	express/app.js, auith.js	
본인 글 작성·수정·삭제	완료	vue/api.js, express/auth.js	
만료·로그아웃·오류 표시	완료	express/auth.js	
DB 데이터 유지	완료	express/users.js, store.js	

구조와 변경 내역

브라우저가 웹 페이지의 블루프린트를 받아 이를 API 서버에 자료를 요청하고, 이를 API 서버가 받아 권한 등을 검증하고, SQLite 데이터베이스와 ORM 으로 통신하며 자료를 찾아 다시 브라우저에게 반환하면 이를 브라우저가 표시하는 연결 구조를 갖습니다.
프론트엔드의 로그인 요청은 이메일과 비밀번호를 바디에 포함해 요청한 뒤, 백엔드가 제공하는 JWT 토큰을 반환받아 세션에 저장하는 것 까지가 요청의 흐름이나, 게시글 수정은 단순 수정 요청 전송 – 백엔드 검증 후 처리 (코드 반환)에 그칩니다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
express/auth.js	인증 관련 임시 함수 구현	인증 관련 함수 실 구현 (본인 작성)	JWT 토큰을 이용한 계정 검증 구현을 위해
express/app.js	Api 라우팅	로깅 수정 (본인 작성)	서버 구동 시 로깅 변경을 위해
vue/api.js	게시글 리스트를 불러오는 등의 API 요청 관련 함수 구현	로그인과 클라이언트 사이드 사용자 정보 검증 구현 (본인 작성)	JWT 토큰 기반 CRUD 요청 전송을 위해
vue/App.vue	웹 페이지 블루프린트	임시 텍스트 제거, 타이틀 변경 (본인 작성)	서비스 이름 변경과 임시 코드 제거를 위해

각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

로컬 실행 기록

Windows 11, version 26H2 / 각 README.md 에 따라 패키지 설치 / 실행 / 프론트엔드 빌드는 frontend-vue 의 dist/에 위치 / SQLite 는 backend-express 의 data/board.sqlite 에 위치.

4173 번 접속과 API 연결 확인 증거 /evidence/connection_api_test.png
서버 재시작 후 데이터 유지 증거 /evidence/data_non_volatility1-3.png

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: express/app.js, express/auth.js:issueToke()

코드 발췌

```
  const payload =
  {
  sub: user.id,
  iss: process.env.JWT_ISSUER,
  aud: process.env.JWT_AUDIENCE
  }
  const token = jwt.sign(payload, process.env.JWT_SECRET, {expiresIn:
`${process.env.TOKEN_TTL}s`})
  return token
```

동작과 응답 코드 설명

입력 검증 및 계정 확인은 app.js 에 구현되어있습니다. (verifyCredentials 함수로 유효 이메일/비밀번호 검증)
각 클레임 중 sub 는 인자로 받은 유저의 id 를 사용, exp, iss, aud 는 env 파일에서 가져와 사용합니다. exp 클레임=유효 시간은 jsonwebtoken 패키지가 제공하는 sign() 함수의 인자로서 명시적으로 페이로드에 포함하지 않았고, env 의 TOKEN_TTL 에 해당하는 초만큼 유효 시간을 설정했습니다.
응답은 유효하지 않은 입력/계정일 경우 401 INVALID_CREDENTIALS, 정상적인 토큰 반환 시 200 OK 입니다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: vue/api.js

코드 발췌

```
function authorizationHeaders() {
  // TODO FE-02: 메모리에 보관한 토큰을 Bearer 헤더로 보내세요.
  return {Authorization: `Bearer ${getSession().accessToken}`};
}
if (!response.ok) {
    clearSession();
    ...
getSession().accessToken = response.accessToken
getSession().user = response.user
```

동작과 응답 코드 설명

로그인 시도 후 200 ok 시 response 에서 토큰과 유저를 추출하여 세션에 저장합니다. 이후 Authorization 에 Bearer 헤더를 통해 요청 시 토큰을 같이 전송하며, 로그아웃 시 세션을 초기화합니다.

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: express/auth.js:currentUserId()

코드 발췌

```
const authHeader = req.headers['authorization'];
  if (!authHeader)
  throw new ApiError(401, "TOKEN_REQUIRED", "JWT 토큰이 존재하지 않습니다.");
  if (!authHeader.startsWith('Bearer '))
  throw new ApiError(401, "TOKEN_REQUIRED", "Bearer 헤더 형식을 확인할 수 없습니다.");
  const token = authHeader.split(' ')[1];
  try {
  const payload = jwt.verify(token, process.env.JWT_SECRET)
  return payload.sub
  } catch(err) {
    throw new ApiError(401, "INVALID_TOKEN", "사용자를 확인할 수 없습니다.");
  // return null
  }
```

동작과 응답 코드 설명

헤더로부터 authroization 추출, 토큰이 존재하지 않거나 헤더 형식을 확인할 수 없다면 401 TOKEN REQUIRED 를 반환하며 아닐 시 jwt.verify 를 통해 토큰을 검증합니다. 이 과정에서 문제 발생 시 401 INVALID_TOKEN 을 반환하며 이외에는 사용자의 id (페이로드의 sub 클레임)을 200 OK 와 함께 반환합니다.

3 4 작성자 권한 검사

파일 / 함수: express/auth.js:assertOwner()

코드 발췌

```
export function assertOwner(userId, authorId) {
  if (userId != authorId){
  throw new ApiError(403, "FORBIDDEN", "올바르지 않은 사용자입니다.")
  }
}
```

동작과 응답 코드 설명

토큰으로 얻어낸 userId 와 요청에 포함된 authorId 를 비교하며, 둘이 일치하지 않을 시 403 FORBIDDEN 을 반환합니다. 이는 유효한 검증이 아니어서 오류가 생긴 401 과는 다르게 검증 상 유효하나 권한이 없어 오류가 생긴 것이며, 이 함수를 통해 글 수정/삭제 시의 작성자 위조를 방지합니다.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: Windows 11, npm 11.21.1, Whale 브라우저

로컬 배포 확인: README 를 따라 설치 / 프론트 빌드 성공 / 백엔드 npm start 로 실행 / 4173 번 접속과 API 연결 확인 증거 /evidence/connection_api_test.png

테스트 일시 및 코드 버전: 26.10.28

토큰 TTL: 일반 테스트 90s / 만료 테스트 3s

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	브라우저/사용자 없음	200 / 해당없음	200 / 해당없음	PASS	/evidence
T02	브라우저/Alice	200 / 해당없음	200 / 해당없음	PASS	/evidence
T03	브라우저/사용자 없음	401 / INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS	PASS	/evidence
T04	브라우저/사용자 없음	401 / TOKEN_REQUIRED	401 / TOKEN_REQUIRED	PASS	/evidence
T05	브라우저/Alice	201 / 해당없음	201 / 해당없음	PASS	/evidence
T06	브라우저/Alice	200, 204 / 해당없음	200, 204 / 해당없음	PASS	/evidence
T07	Postman/Bob	403 / FORBIDDEN	403 / FORBIDDEN	PASS	/evidence
T08	Postman/Bob	403 / FORBIDDEN	403 / FORBIDDEN	PASS	/evidence
T09	Postman/Bob	401 / INVALID_TOKEN	401 / INVALID_TOKEN	PASS	/evidence
T10	Postman/Bob	401 / TOKEN_EXPIRED	401 / TOKEN_EXPIRED	PASS	/evidence
T11	브라우저/사용자 없음	401 / TOKEN_REQUIRED	401 / TOKEN_REQUIRED	PASS	/evidence
T12	Postman/Bob	400 / BAD_INPUT	400 / BAD_INPUT	PASS	/evidence
T13	Postman/Bob	400 / BAD_INPUT	400 / BAD_INPUT	PASS	/evidence
T14	Postman/Bob	404 / NOT FOUND	404 / NOT FOUND	PASS	/evidence
T15	브라우저/사용자 없음	200 / 해당없음	200 / 해당없음	PASS	/evidence
T16	브라우저/사용자 없음	200 / 해당없음	200 / 해당없음	PASS	/evidence
T17	브라우저/Bob	401 / TOKEN_REQUIRED	401 / TOKEN_REQUIRED	PASS	/evidence

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	**200**	/evidence
새 글 등록	**201 및 Location**	**201 및 Location**	/evidence
본인 글 삭제	**204 및 빈 본문**	**204 및 빈 본문**	/evidence
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	**400 BAD_INPUT**	/evidence
잘못된 계정	**401 INVALID_CREDENTIALS**	**401 INVALID_CREDENTIALS**	/evidence
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	**401 TOKEN_REQUIRED**	/evidence
변조·클레임 오류	**401 INVALID_TOKEN**	**401 INVALID_TOKEN**	/evidence
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	**401 TOKEN_EXPIRED**	/evidence
다른 작성자의 변경	**403 FORBIDDEN**	**403 FORBIDDEN**	/evidence
없는 글	**404 NOT_FOUND**	**404 NOT_FOUND**	/evidence

테스트 증거

로그인과 토큰 전송

화면 캡쳐 : /evidence/test1-1.png, /evidence/test1-2.png

다른 작성자의 수정과 삭제

화면 캡쳐 : /evidence/test2-1.png, /evidence/test2-2.png

토큰 변조와 만료

화면 캡쳐 : /evidence/test3-1.png, /evidence/test3-2.png

자동 검사와 로컬 배포 증거

자동 검사 결과 증거 /evidence/check_api.png
python .\tools\check_api.py 로 자동 검사를 실행할 수 있으며, 백엔드 서버에 대한 API 검사를 자동으로 진행합니다.

빌드 성공 로그 증거 /evidence/build_success.png
4173 번 접속과 API 연결 확인 증거 /evidence/connection_api_test.png
서버 재시작 후 데이터 유지 증거 /evidence/data_non_volatility1-3.png

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: 미사용

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
jsonwebtoken (https://github.com/auth0/node-jsonwebtoken#readme)	express/auth.js	함수 작성

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

타인이 임의로 authorId 를 조작한 요청을 보내 남을 사칭할 수 있는 문제가 존재한다.
현재의 코드는 이러한 요청 시 JWT 토큰을 같이 받고, JWT 토큰에 포함된 유저 정보를 교차 검증하는 것으로 이러한 문제를 방지한다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

수정 버튼을 숨긴다고 하여 브라우저 또는 외부 툴을 이용하여 요청을 보낼 수 없다는 것을 의미하지 않기 때문이며, 이 때문에 현재 auth.js 의 assertOwner() 함수가 권한을 검사한다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 단순히 암호화 되어있는 토큰을 복호화 하는 것을 칭하며, verify 는 이에 더해 페이로드 내의 정보들의 유효성 검증 (올바른 사용자인지? 만료된 토큰은 아닌지?) 이 포함된 개념이며, 현재 코드에선 토큰이 만료되진 않았는지, 또한 사용자 대조의 두 검증을 수행한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

현재 코드에서 로그아웃 시 프론트엔드는 명시적으로 세션 내의 정보를 초기화하며, 이는 새로고침 시에도 동일하게 작동한다. 토큰 만료 시 토큰 재발급이 필요하며 정보가 바뀌진 않으나 같은 토큰임에도 요청 시 사용할 수 없게 된다. 때문에 이미 복사된 토큰, 탈취한 토큰 등은 유효 시간 전까지 사용이 가능하고 토큰을 만료하는 것 외에는 이를 막을 수 없기 때문에 통상적으로 JWT 토큰을 과하게 긴 사용 시간을 설정하지 않는다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

토큰 생성 시 exp 클레임을 명시적으로 페이로드 내에 추가하여 구현하려 했으나 자료형 불일치 등의 문제로 제대로 작동하지 않았고, 공식 jsonwebtoken 문서에서 제공하는 expiresIn 옵션을 이용하여 넣는 것으로 해결했습니다.

현재 남은 문제와 다음에 개선할 점: 없음

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출